Months 13 to 15: LLMs

Quick reference · Senior Data Scientist roadmap · Prompting with structured output, retrieval-augmented generation and evaluating LLM systems.

B20 Prompting and structured output

B20-L1 Prompts as data, system prompt, messages and few-shot turns

Instructions in the system prompt, data in tagged user messages, examples as earlier turns, and a version on every result.

- **The two inputs to an LLM call**: A system prompt with instructions, and a list of messages with role user or assistant. The first message is a user turn.
- **Where respondent text goes**: In a user message inside tags such as <answer>...</answer>, with tag text removed from the answer first.
- **Few-shot examples**: Earlier turns: the example answer as a user message, then the wanted label as an assistant message.
- **What every model result records**: The prompt version and the model ID, with the prompt loaded from a versioned file.

B20-L2 Structured output, validate every reply and retry once

Validate every reply against a model of what you need, retry once with the error, and send the rest to review.

- **Parse and validate a model reply in one step**: Model.model_validate_json(reply). Broken JSON and bad fields both raise ValidationError.
- **Structured output in anthropic 2.4.0**: client.messages.parse(..., output_format=PydanticModel), then response.parsed_output.
- **When a reply fails validation**: Add the reply as an assistant turn and the error as a user turn, retry, and stop after a fixed number of attempts.
- **When every attempt fails**: Store needs_review with the raw reply. Never guess a label.

B20-L3 Model calls in a pipeline, stop reasons, caching and run logs

Inject the model, reject replies that did not finish, cache on model plus prompt version plus messages, and log every call.

- **stop_reason values to reject**: Anything but end_turn: max_tokens means the reply was cut off, refusal means the model declined.
- **Cache key for model replies**: sha256 of json.dumps({model, prompt_version, messages}, sort_keys=True), as hexdigest().
- **What the anthropic 2.4.0 SDK retries**: Connection errors and status 408, 409, 429 and 500 or above, twice by default (max_retries=2).
- **One run-log record per model call**: run_id, UTC timestamp, model, prompt version, cache hit, stop reason, input and output tokens.

B21 Retrieval-augmented generation

B21-L1 Chunk documents for retrieval

Split on structure, then by size with overlap. Keep every word, and keep the metadata that citations need.

- **The three steps of RAG**: Retrieve relevant chunks, put them in the prompt, and answer from them alone with citations.
- **Sliding window starts**: range(0, max(len(words) - overlap, 1), size - overlap). Every word covered, no window of repeats.
- **Chunk size trade-off**: Small chunks match precisely but lose context. Large chunks keep context but blur the match. Test sizes on an evaluation set.
- **Metadata on every chunk**: Document ID and version, section, position and a stable chunk ID, for citations, filters and re-indexing.

B21-L2 Retrieve with embeddings, keywords and rank fusion

Filter, score by cosine similarity, add a keyword ranking, fuse with RRF, and report when nothing matches well.

- **Top k by cosine similarity with NumPy**: Divide dot products by both lengths, then np.argsort(-scores)[:k]. Guard zero lengths.
- **Reciprocal rank fusion**: Each ranking gives a chunk 1 / (k + rank), ranks from 1, summed across rankings. k = 60 in the original paper.
- **Why add keyword search**: Embeddings blur exact tokens such as wave codes and variable names. Keyword ranking matches them.
- **Before ranking, and after**: Filter to chunks the question may use. If the best score is low, report that nothing matched.

B21-L3 Grounded answers, context budgets and citation checks

Tag each source with its ID, fill the budget in rank order without cutting chunks, and check every citation against what you sent.

- **Rules for a grounded answer**: Answer only from the sources, cite [id] after each claim, use the fixed refusal when the sources lack the answer, and treat source text as data.
- **Filling the context budget**: Add tagged chunks in rank order, check used + n before adding, and stop at the first overflow. Never cut a chunk.
- **Counting tokens in anthropic 2.4.0**: client.messages.count_tokens(model=..., system=..., messages=...).input_tokens.
- **Citation check**: re.findall(r"[([^]]+)]", answer). Flag IDs you never sent, and answers without a citation unless they are the refusal.

B22 Evaluating LLM systems

B22-L1 Evaluate retrieval, recall at k and MRR on a gold set

Score retrieval apart from generation, on a versioned gold set, with recall at k and MRR, and compare settings on the same questions.

- **Recall at k**: len(set(ranked[:k]) & relevant) / len(relevant). With one relevant section per question, its mean is the hit rate.
- **Mean reciprocal rank**: Mean over questions of 1 / rank of the first relevant result, ranks from 1, or 0 when none appears.
- **Gold set rules**: Real questions, relevance marked by section, versioned, and every failed question added.
- **Repeated sections in a ranking**: list(dict.fromkeys(section_ids)) keeps each at its first position.

B22-L2 Evaluate answers, exact checks, token F1 and LLM judges

Exact checks first, token F1 for trends, and a validated, structured judge for free text, run in both orders for pairs.

- **Order of answer checks**: Exact checks first, then token F1 for trends, then a validated LLM judge for free text.
- **Token F1**: Shared words with Counter &; p = shared / reply words, r = shared / reference words, F1 = 2pr / (p + r).
- **LLM judge biases (Zheng et al., NeurIPS 2023)**: Position, verbosity and self-enhancement bias. Swap the order for pairs and call disagreements ties.
- **Before trusting a judge**: Compare it with human grades on a sample, criterion by criterion, and re-check after any prompt or model change.

B22-L3 Regression evals and release gates for prompt changes

Compare per item, repeat noisy items, write the gate first, and never let a higher average hide a broken critical item.

- **Fixed and broken items**: Fixed: fail in the baseline, pass in the candidate. Broken: the reverse. Read every broken item.
- **Majority over repeated runs**: Pass when sum(runs) > len(runs) / 2. Unstable when 0 < sum(runs) < len(runs).
- **A release gate**: Written before the run: every critical item passes, at most N items break, and the pass rate drops by no more than a set share.
- **What to log for each eval run**: Model, prompt version, eval set version, verdicts per item and token counts.